

Factory video in, streamed work instructions out

An API-first service that turns a shop-floor recording into ordered, timestamped operator instructions — each step paired with its own video clip — and pushes them to the client as they are produced rather than at the end of the run.

SERVICE NAME	BACKEND	FRONTEND	INTELLIGENCE
qsaid-wi · v1.0.0	Python 3.11 · FastAPI · uvicorn	React 18 · Vite (separate deploy)	Azure OpenAI GPT-4o + Whisper
STATE & ARTIFACTS	RUNTIME TARGET	BACKEND CODE	TRANSPORT
Azure Blob Storage	Azure Web App (Linux container)	~4,200 LOC Python	REST for control, WebSocket for results

CONTENTS

01	What the system does	02	Design goals and the constraints they set
03	Component model	04	Job lifecycle
05	Analysis pipeline and concurrency	06	From frames to steps
07	Streaming contract	08	Data model and storage
09	External dependencies and cost shape	10	Deployment topology
11	Failure handling and degradation	12	Security posture
13	Scale ceiling and known gaps	14	Recommended next moves

What the system does

An operator or process engineer uploads a video of a task being performed. The service samples the video into frames, transcribes the narration, asks a vision model what the worker is doing in each frame, folds those two timelines into a set of non-overlapping time windows, and writes one operator-ready instruction per window. Every instruction carries its start and end timecode, the instruments seen or mentioned, and a URL to a video clip cut to exactly that window.

The distinguishing behaviour is *when* the client sees results. Instructions are not returned as one document at the end of processing — each step is pushed over a WebSocket the moment it is ready, in strict order, so a UI can render a growing procedure while the back half of the video is still being analysed.



FIG 1 Control and results flow through the API; video bytes for playback do not. Clips are uploaded once to Blob Storage and handed to the browser as time-limited SAS URLs, keeping the API off the media serving path.

Design goals and the constraints they set

Four goals shape most of the structure. Each buys something and costs something, and the costs explain design choices that would otherwise look odd.

GOALS → ARCHITECTURAL CONSEQUENCES

GOAL	HOW IT IS MET	WHAT IT COSTS
Perceived latency near zero	Job id returned before any bytes are sent; steps streamed individually; report and artifact upload deferred until after the last step	Two round trips to start a job; the <code>complete</code> event arrives before artifacts exist
Full video coverage	Timeline gaps are filled and idle windows are dropped only when coverage survives; the first window is forced to <code>0.0s</code> and the last to the video duration	Some low-value steps survive; step count tracks video length, not task complexity
Grounded instructions	One focused GPT call per window, given only that window's narration and visible actions, in JSON mode at temperature 0.2, with prior steps as context	One model call per step; wording quality depends on per-window evidence, not a whole-video view
Survive a dropped connection	Every step is persisted to blob as it streams; a reconnecting client is replayed the saved steps before being subscribed to live events	A blob write on the hot path of every step emission

Component model

The backend is one process with a clean split between the HTTP edge, job state, orchestration, and the domain logic that turns observations into instructions. The domain logic is by far the largest piece — `wi_steps.py` alone is about a third of the backend.

MODULES AND RESPONSIBILITIES

MODULE	LOC	RESPONSIBILITY
<code>api/main.py</code>	42	App assembly, CORS, <code>/</code> and <code>/health</code>
<code>api/routes/jobs.py</code>	380	REST + WebSocket endpoints, file validation, chunked upload to a temp file, background task spawn, status publishing
<code>api/models.py</code>	28	Pydantic request and response contracts
<code>api/services/job_manager.py</code>	179	Job records, per-connection subscriber queues, blob persistence of metadata and steps, temp work-dir registry
<code>api/services/pipeline_runner.py</code>	509	Orchestration: monotonic status emitter, ordered step emitter, clip upload futures, deferred finalisation
<code>api/services/wi_steps.py</code>	1308	Domain core: segment construction, speech-based splitting, gap filling, idle filtering, dedup, per-segment instruction generation, overlap enforcement
<code>api/services/step_enricher.py</code>	44	Instrument extraction and the wire shape of a step event
<code>api/services/video_clips.py</code>	73	ffmpeg clip cutting with an audio-free fallback
<code>api/services/blob_storage.py</code>	137	Blob upload/download, existence checks, listing, SAS generation
<code>pipeline.py</code>	243	The five-stage analysis run; also holds the legacy whole-video instruction writer
<code>sop_stateless.py</code>	328	Streaming frame analysis: worker pool, near-duplicate frame skip, disk cache, run consolidation
<code>report_builder.py</code>	792	Interactive HTML report, VTT/transcript writers, timestamp parsing shared with the domain core
<code>vision_analysis.py</code>	152	Frame prompt, garbage-response detection, retry with a stricter prompt
<code>audio_transcription.py</code>	123	Audio extraction and Whisper call with three response-format fallbacks
<code>key_frame_generator.py</code>	60	Interval frame extraction into a bounded queue; video duration
<code>azure_client.py</code>	120	Client construction, endpoint normalisation, DNS reachability preflight

Frontend

The UI is a separate Vite/React application that talks to the API over `VITE_API_BASE_URL`. All streaming state lives in one hook, `useJobStream`, which owns the create → subscribe → upload sequence and keeps steps sorted by index; components (`VideoUpload`, `JobProgress`, `StepCard`, `StepDetail`) are presentational. A 15-second health poll drives an online indicator, and completed jobs are recorded in browser storage as a local history list.

Job lifecycle

The primary flow deliberately splits job creation from the upload. Creating the job is a small JSON call that returns an id immediately, so the client can open its WebSocket *before* sending a single byte of video. The upload then reports its own progress on that already-open channel, and the pipeline continues on the same channel without any reconnect.

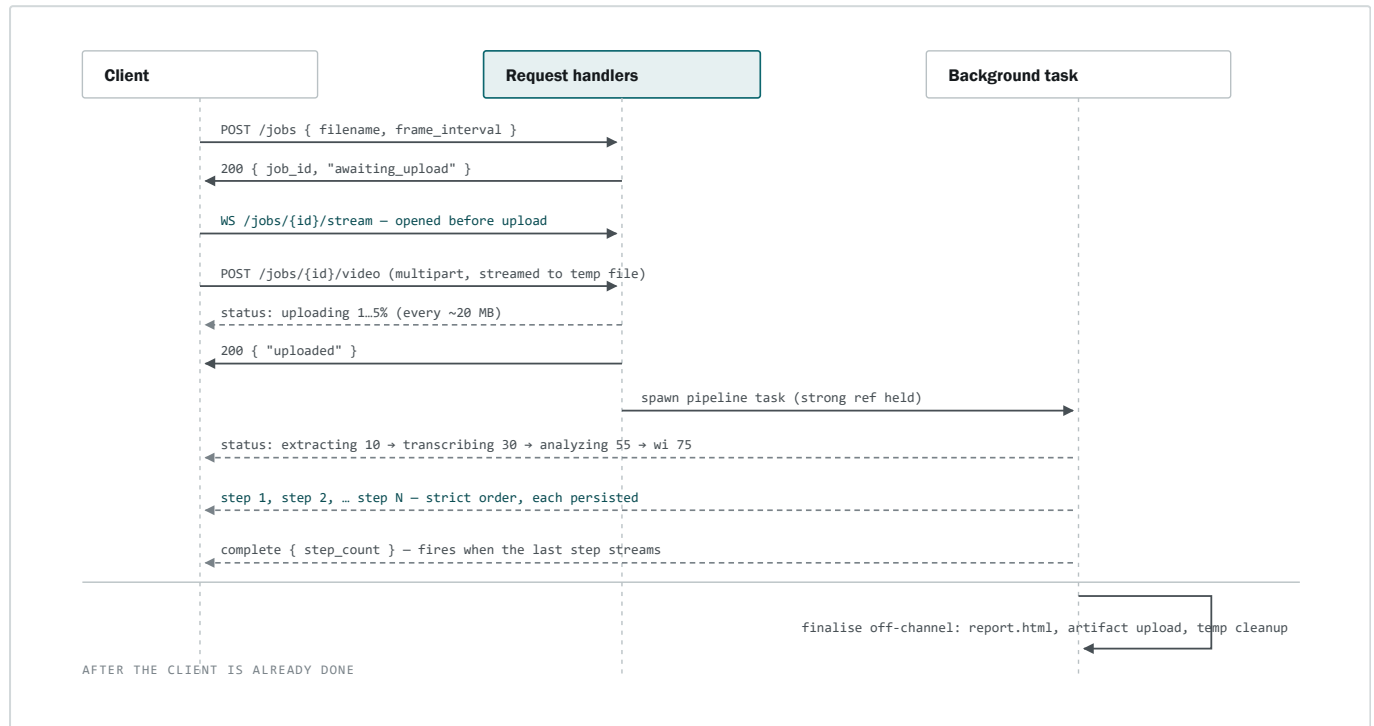


FIG 2 One socket carries upload progress, pipeline status, every `complete` is published when streaming ends, so the report and artifact uploads happen after the client considers the job finished. Note the ordering below the rule:

CONSEQUENCE WORTH KNOWING

Because completion is announced early, the `complete` event on a live connection carries only `step_count`. The variant carrying `report_url`, `work_instructions_url` and `metadata_url` is sent when a client connects to a job that is *already* completed — the replay path. A client that wants artifact links for the job it just watched must re-read `GET /api/v1/jobs/{id}` or reconnect the socket.

Status machine

Status is a single string on the job record, mirrored to every subscriber and persisted to `metadata.json`. Progress is a percentage fixed per stage, and a monotonic guard drops any update that would move it backwards — necessary because stages report from worker threads that can finish out of order.

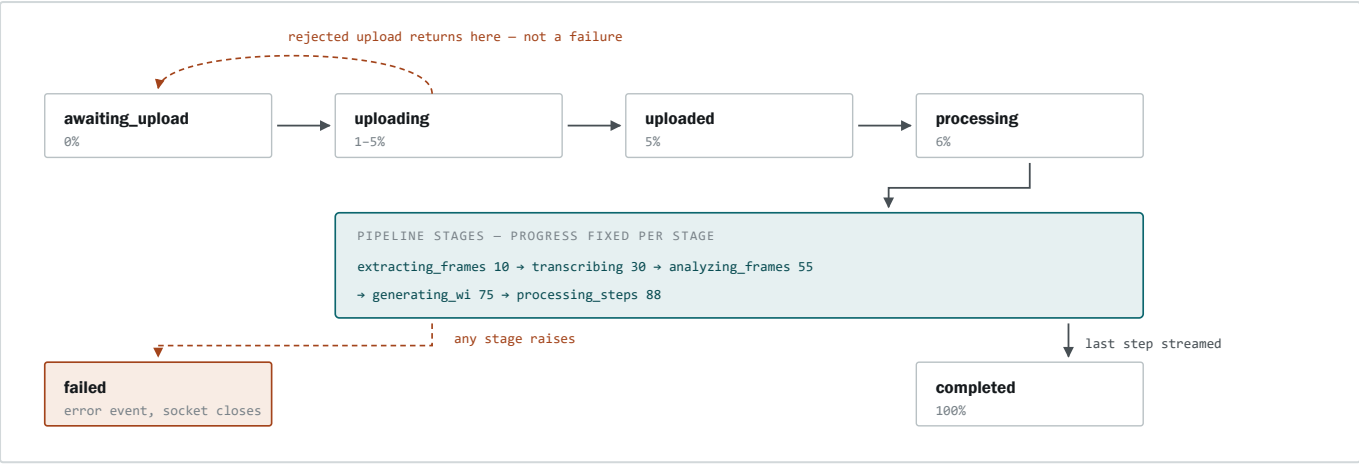


FIG 3 The recoverable edge is the one worth noticing: an unsupported extension, `awaiting_upload` with a message, leaving the socket open so the client can retry the same job id.

Analysis pipeline and concurrency

The pipeline runs on threads driven from the event loop, and the concurrency is where most of the wall-clock saving lives. Frame extraction, transcription, and vision analysis all run at the same time: extraction pushes filenames onto a bounded queue of 64, and the vision pool consumes from that queue rather than waiting for extraction to finish.

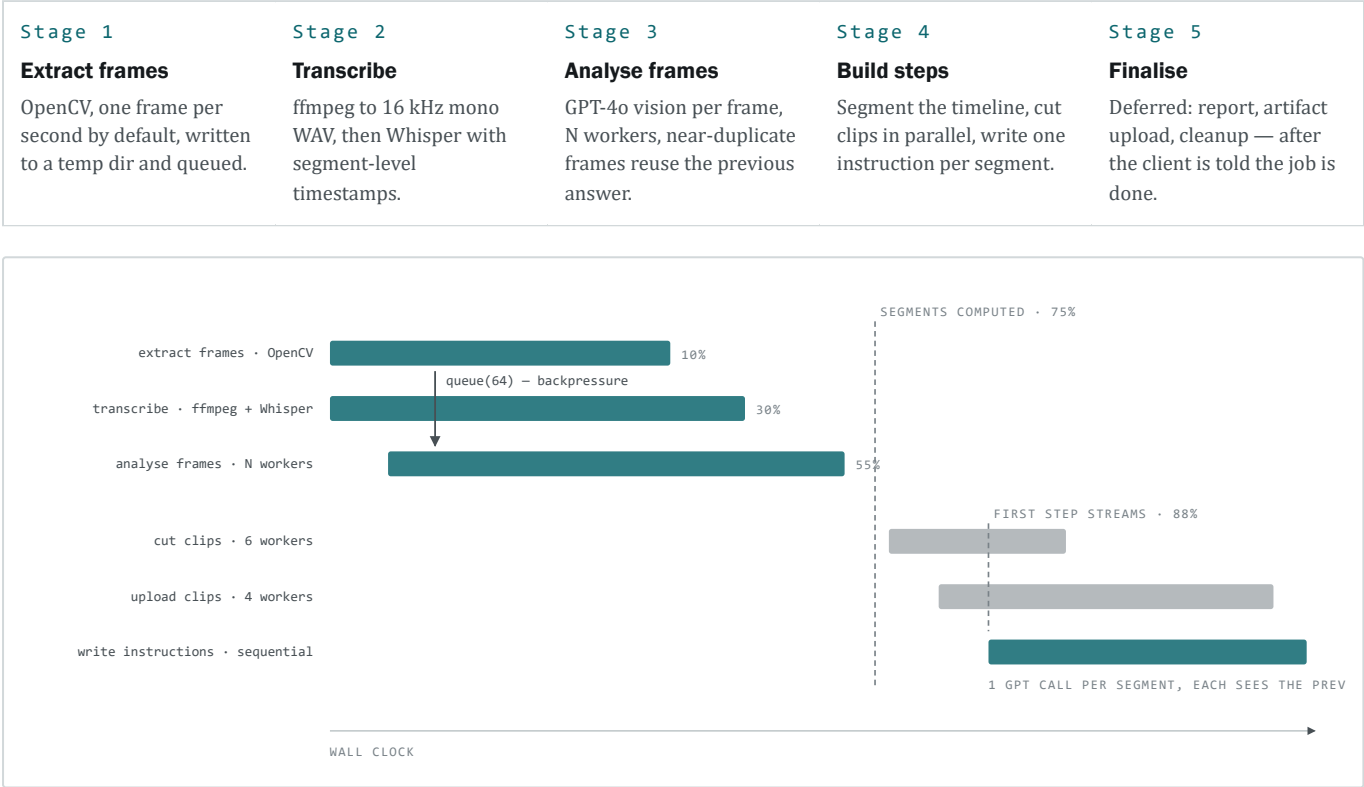


FIG 4 Two overlapping fan-outs separated by one barrier. Segments cannot be computed until both the transcript and the full visual sequence exist; after the barrier, clip cutting and uploading run ahead of the sequential instruction writer, so a clip URL is usually ready by the time its step is emitted.

Bridging threads and the event loop

Analysis runs in thread pools while all client-facing publishing happens on the event loop, so two small components own the boundary.

- **Monotonic status emitter** — worker threads call `loop.call_soon_threadsafe` to enqueue a stage update; a single drain task applies them in order and discards any whose progress is lower than the last one published.
- **Ordered step emitter** — completed segments are offered via `run_coroutine_threadsafe`, buffered by segment index behind an `asyncio.Lock`, and drained only when the next expected index is present. The client therefore always sees 1, 2, 3... with no gaps, whatever order work finished in.

CONCURRENCY AND TUNING KNOBS

Knob	Default	Where it applies
frame_interval	1.0 s	Per job; clamped to a 1 s floor. Directly sets vision call volume
vision_workers	8	Parallel GPT-4o vision calls; the main memory pressure point
analyze_every	1	Sample every Nth extracted frame — the cheap quality/cost dial

KNOB	DEFAULT	WHERE IT APPLIES
frame queue depth	64	Backpressure between extraction and analysis
WI_WORKERS	6	Parallel ffmpeg clip cuts
UPLOAD_WORKERS	4	Parallel clip uploads to blob
upload chunk	1 MB	Read size while streaming the video to a temp file
step window	8–32 s	Target segment length before splitting or merging

From frames to steps

This is the part of the design that does the real work, and it is deliberately mostly deterministic. Only two places call a model: one vision call per frame at the front, and one writing call per segment at the end. Everything between them is code — grouping, merging, splitting, gap filling, and filtering — which is why step boundaries are reproducible and why a model failure degrades to a heuristic instead of losing the step.

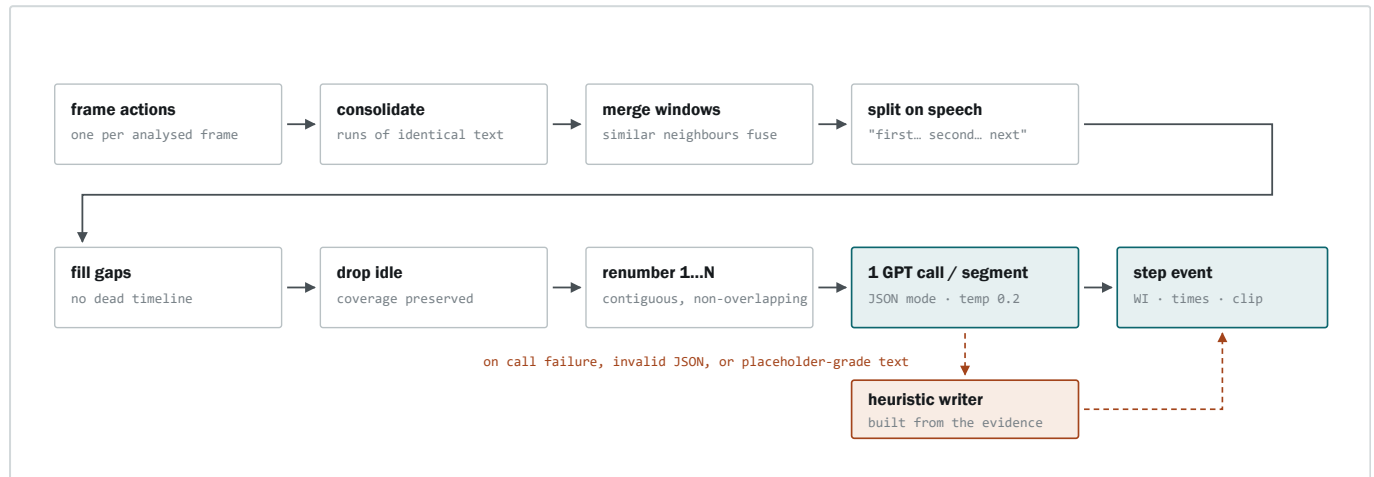


FIG 5 Seven deterministic transforms bracketed by two model calls. The fallback path matters: a step is never dropped because generation failed — it is rewritten from the same narration and visible-action evidence by code.

What each transform is for

- **Consolidate** — identical consecutive frame descriptions collapse into one run with a start and end frame, which is what turns per-second observations into candidate windows.
- **Merge** — adjacent windows whose descriptions are similar past a 0.68 similarity threshold fuse, so a single continuous action does not become six steps.
- **Split on speech** — enumeration cues in the narration (*first*, *second*, *next*...) split a long window, because the narrator's own list structure is better evidence of step boundaries than visual similarity.
- **Fill gaps** — any uncovered stretch of timeline gets a window, so the procedure never silently skips video.
- **Drop idle** — windows with no usable observation and no speech are removed, but only where removing them does not open a coverage hole.
- **Enforce non-overlap** — a final pass clamps boundaries so step n ends no later than step $n+1$ begins, and the last step ends at the video duration.

GENERATION PROMPT DISCIPLINE

The per-segment call is tightly fenced: it sees only that window's narration and visible actions, is told to preserve the narrator's wording, is forbidden from referencing the video or inventing safety notes, and must return `{"WI": "...", "instruments": []}`. Output is capped at 320 tokens. Weak or placeholder-grade responses are detected and replaced by the heuristic writer. Instrument names are also parsed out of an `Instruments used:` line when the model emits one inline.

Streaming contract

The API surface is small. Control is REST; results are WebSocket only.

HTTP AND WEBSOCKET ENDPOINTS

VERB	PATH	PURPOSE
GET	/health	Liveness; the UI also uses the service name to detect a wrong process on the port
POST	/api/v1/jobs	Create a job from JSON metadata; returns <code>job_id</code> immediately
POST	/api/v1/jobs/{id}/video	Upload the video for an existing job; guarded by an atomic status transition so a job accepts exactly one upload
POST	/api/v1/jobs/upload	Legacy single-request create-and-upload; no id until the whole body has arrived
GET	/api/v1/jobs/{id}	Job record plus every persisted step — the polling fallback
WS	/api/v1/jobs/{id}/stream	Event stream; closes with code 4004 for an unknown job

WEBSOCKET EVENT TYPES

TYPE	PAYLOAD AND MEANING
status	<code>status</code> + <code>progress</code> , optionally <code>message</code> on a recoverable upload rejection
step	<code>step_index</code> , <code>WI</code> , <code>timestamp_start/end</code> , <code>video_clip_url</code> (SAS), <code>instruments[]</code>
complete	<code>step_count</code> ; the replay path additionally carries metadata, instructions and report SAS URLs
error	<code>message</code> ; terminal — the socket closes after it
ping	Emitted after 30 s of silence to keep intermediaries from dropping an idle socket

Reconnect semantics

On connect the handler branches on stored status. A completed job replays all saved steps then sends the artifact-bearing `complete` and closes. A failed job sends `error` and closes. A live job replays saved steps, subscribes to the live queue, sends a current `status`, and de-duplicates by step index so a step that arrives both from the replay and the live queue is only delivered once. Multiple concurrent viewers of the same job each get their own queue and all receive every event.

Data model and storage

There is no database. Blob Storage is the system of record, and the job id is the partition key. In-process dictionaries act as a write-through cache in front of it.

BLOB LAYOUT UNDER THE CONFIGURED CONTAINER

PATH	CONTENTS
jobs/{id}/metadata.json	Status, progress, timestamps, original filename, pipeline options, step count, artifact paths, error
jobs/{id}/source/video.ext	The uploaded video, copied up during finalisation
jobs/{id}/steps/step_NNN.json	One file per emitted step event — the replay source
jobs/{id}/clips/segment_NNN.mp4	Per-step clips from the streaming path
jobs/{id}/artifacts/...	work_instructions.txt , report.html , transcript.txt , transcript.vtt , transcript_segments.json , visual_sequence.json , timeline.json

IN-PROCESS STATE

Three dictionaries on a single `JobManager` instance: cached metadata per job, a list of subscriber queues per job, and the temp work directory per job. Metadata writes are mostly fire-and-forget tasks so frequent progress ticks never block on a blob round trip; step writes are awaited, since a step that is not persisted cannot be replayed.

NAMING INCONSISTENCY

The streaming path writes clips as `clips/segment_NNN.mp4` while the fallback helper and the README both use `clips/step_NNN.mp4`. Both work — the SAS URL is generated from whichever path was written — but anything that enumerates clips by name (a cleanup job, an export, a migration) needs to know both conventions.

External dependencies and cost shape

Model spend scales with video length, not with task complexity, and it is dominated by the vision stage.

PER-JOB EXTERNAL CALLS

DEPENDENCY	CALLS PER JOB	NOTES
GPT-4o vision	$\approx \text{duration} \div \text{interval}$	One per analysed frame, plus a retry when the response looks like OCR junk. Near-duplicate frames (0.97 similarity) and a disk cache remove some calls
GPT-4o writer	1 per segment	Sequential, so it also sets the tail latency of streaming
Whisper	1 per job	Whole audio track in one request — the practical duration ceiling of the design
Blob Storage	$O(\text{steps}) + \text{artifacts}$	One metadata write per status change, one step write per step, one upload per clip and artifact
ffmpeg (local)	1 + segments	Audio extraction, then one re-encode per clip — CPU-bound work on the app instance
OpenCV (local)	1 decode pass	Frame sampling and duration probing

A ten-minute video at the default one-frame-per-second yields roughly 600 vision calls and, depending on segmentation, 20–60 writer calls. The two cheapest levers are `frame_interval` and `analyze_every`; both trade description density for cost and both are per-job parameters, so they can be tuned by video type rather than globally.

Deployment topology

The backend deploys as a single container to an Azure Web App; the UI deploys separately and is pointed at the API host by build-time configuration. CI builds and pushes on every merge to `main`.

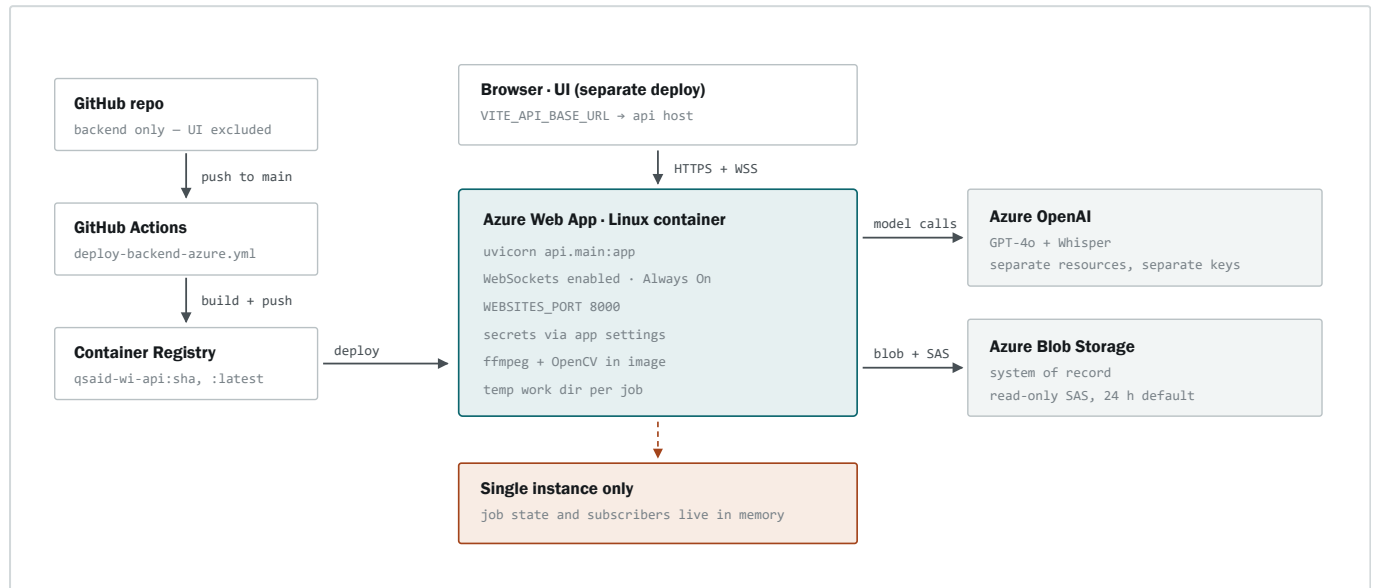


FIG 6 A conventional container pipeline with one hard constraint attached: because subscriber queues, cached job metadata and temp work directories are process-local, the plan can scale up but not out until that state moves outside the process.

Platform settings that are not optional: WebSockets must be enabled on the Web App, Always On must be on so background pipeline tasks are not suspended when the app idles, and the maximum request body size must be raised to accept large videos. `CORS_ORIGINS` must be set to the UI origin.

Failure handling and degradation

The system prefers a degraded result over a failed job. Most external failures reduce quality rather than aborting the run.

FAILURE MODES AND RESPONSES

FAILURE	RESPONSE	JOB OUTCOME
No audio track	Skip transcription, continue on visual evidence only	Completes, thinner instructions
Whisper call fails	Warn, continue with an empty transcript	Completes
Whisper rejects the response format	Retry without timestamp granularity, then as plain text	Completes, may lose segment timing
Vision returns OCR-style junk	Detected by heuristic, retried with a stricter prompt, then a generic action stands in	Completes
Single frame analysis error	Logged and skipped	Completes
Vision endpoint unreachable 3×	Abort the stage with a configuration-oriented message	failed
Writer call fails or returns weak text	Heuristic writer builds the step from the same evidence	Completes
Clip cut fails with audio	Retry video-only, then continue with no clip URL for that step	Completes, step has no clip
Clip upload fails	Step is emitted with <code>video_clip_url: null</code>	Completes
Artifact or report stage fails	Recorded as <code>artifact_error</code> on the job record; the client was already told the job completed	Completes, no report
Upload rejected or interrupted	Status returns to <code>awaiting_upload</code> with a message; socket stays open	Retryable on the same id
No segments produced	Explicit error	failed

OBSERVABILITY

Diagnostics are `print` statements to stdout, collected as container logs. There are no metrics, no traces, no structured log fields, and no per-stage timing beyond what can be inferred from log timestamps. For a demo this is adequate; for anything with an SLA it is the first gap to close, because the dominant cost and latency contributor is a per-frame external call that is currently unmeasured.

Security posture

State this plainly, because the current posture is appropriate for a demo and not for production data.

- **No authentication or authorisation** on any endpoint. Anyone who can reach the host can create jobs and spend model budget.
- **The job id is the only capability.** It is a UUID4, so it is unguessable, but knowing one grants full read access to that job's steps and clips.
- **Storage account key in application settings.** SAS URLs are signed with the account key, which means the running process holds credentials for the whole account rather than a scoped identity.
- **SAS URLs are read-only and time limited** — 24 hours by default — but once issued they are bearer tokens in the client's hands and cannot be revoked individually.
- **CORS defaults to *** and must be narrowed per environment.
- **No retention policy.** Source videos, clips and transcripts persist indefinitely. For shop-floor footage this is a records-management question as much as a security one.

Scale ceiling and known gaps

WHERE THE CURRENT DESIGN STOPS

LIMIT	CAUSE	EFFECT WHEN REACHED
One instance	Subscriber queues, metadata cache and work dirs are process-local	A second replica serves sockets for jobs it is not running; clients see no events
Restart loses in-flight jobs	Pipeline tasks are in-process with no durable queue or lease	Job is stuck at its last persisted stage forever; no resume, no timeout
No cancellation	No handle exposed on the spawned task	A wrong 40-minute upload runs to completion and bills for it
CPU contention	One ffmpeg re-encode per clip, plus OpenCV decode, on the web instance	Clip cutting competes with request handling on the same plan
Whole-file Whisper call	Audio is sent as one request	Long videos hit request size and duration limits with no chunking path
Sequential writer	Each step is generated with prior steps as context	Streaming tail latency grows linearly in segment count
Whole-blob reads and writes	<code>upload_file</code> reads the file fully into memory	Memory spikes proportional to the largest clip or the source video
New client per blob call	A <code>BlobServiceClient</code> is constructed per operation	Avoidable connection setup on a hot path
Dead code	<code>api/services/stream_upload.py</code> is complete but never imported	A streaming-multipart path exists on paper only; routes buffer to a temp file instead

Recommended next moves

Ordered by how much they unlock relative to the work involved.

1. **Move fan-out off process memory.** Replace the in-memory subscriber dictionary with Redis pub/sub or Azure Web PubSub. This is the single change that turns a one-instance demo into something that can scale out, and it touches only `JobManager`.
2. **Put pipeline runs on a durable queue.** A Service Bus or Storage Queue message with a lease, and workers separate from the web tier, gives restart survival, cancellation, retry and the ability to size CPU for ffmpeg independently of request handling.
3. **Add authentication and scope job access.** Even a simple bearer token plus an owner field on the job record removes the "unguessable id is the security model" problem.
4. **Switch to a managed identity and user-delegation SAS.** Removes the account key from application settings and makes issued URLs revocable by rotating the delegation key.
5. **Instrument the vision stage.** Per-stage timings, call counts, cache hit rate and duplicate-skip rate. It is the dominant cost centre and currently invisible.
6. **Chunk the Whisper call.** Split audio on silence with overlap and stitch segment timings, to lift the practical video-length ceiling.
7. **Send artifact URLs on the live socket.** Publishing a second terminal event (or an `artifacts` event) when finalisation completes removes the need for clients to poll after they have already been told the job is done.
8. **Add a retention policy** on the container, and a delete endpoint for a job and all its blobs.

ASSESSMENT

The domain logic is the strong part of this system: the timeline segmentation is deterministic, coverage-preserving and independently testable, and every model call has a defined fallback. The weak part is entirely operational — state lives in one process, work has no durable home, and the most expensive stage is unmeasured. Those are additive fixes at the edges rather than a redesign, which is a good position to be in.